Name: Hamza Hashmi

Roll no: A-35

Branch:- AI&DS(A)

Experiment:-06

AIM:- Implementing Web Scraping in Python with BeautifulSoup

Theory :-

1. What is web scraping? Role of Requests library

Web scraping is the process of extracting data from websites automatically using programs.

Role of Requests library:

-Sends HTTP requests to web pages.

-Retrieves the HTML content of a webpage.

-Acts as the first step in scraping (getting the page data)

Example:-

```
import requests
url = "https://example.com"
response = requests.get(url)
print(response.text)
```

2. What is BeautifulSoup? How does it help?

BeautifulSoup is a Python library used to parse HTML and XML documents.

How it helps:

Toggle Gemini

-Converts raw HTML into a structured format (parse tree).

-Makes it easy to search and extract elements

-Handles poorly formatted HTML.

Example:-

```
from bs4 import BeautifulSoup
html = "<html><body><h1>Hello</h1></body></html>"
soup = BeautifulSoup(html, "html.parser")
print(soup.h1.text)
```

3. Difference between Web Scraping and Web Crawling

Feature	Web Scraping	Web Crawling
Primary Goal	Extract specific data from web pages	Discover and index web pages
Mechanism	Focuses on parsing content of individual pages	Follows links to navigate between pages
Output	Structured data (e.g., CSV, JSON)	A list of URLs or an index of web content
Scope	Targeted extraction from selected pages	Broad discovery across many pages/websites
Tools Used	BeautifulSoup, Scrapy, Selenium	Spiders, bots, search engine crawlers
Analogy	Picking specific fruit from a tree	Exploring the entire orchard to find all trees

4. Commonly used methods in BeautifulSoup

Some important methods:

-find() ⟶ Finds first matching element.

-find_all() ⟶ Finds all matching elements.

-get_text() ⟶ Extracts text.

-get() ⟶ Gets attribute value

-select() ⟶ CSS selector-based search

Example:-

```
soup.find('p')
soup.find_all('a')
soup.get_text()
```

5. Ethical and Legal Considerations

When doing web scraping, you must follow rules:

Ethical considerations:

-Respect website policies (check robots.txt).

-Avoid overloading servers (use delays).

-Do not scrape sensitive/private data

Legal considerations:

-Follow terms of service of websites.

-Avoid copyrighted or restricted content.

-Ensure data is used responsibly

-List item

```
import requests
from bs4 import BeautifulSoup
# Step 1: Define the URL
url = "https://example.com"
# Step 2: Send HTTP request
response = requests.get(url)
# Step 3: Check if request was successful
if response.status_code == 200:
print("Successfully fetched the webpage!\n")
# Step 4: Parse HTML content
soup = BeautifulSoup(response.text, "html.parser")
# Step 5: Extract all links (anchor tags)
links = soup.find_all('a')
print("Links found on the webpage:\n")
for link in links:
text = link.get_text()
href = link.get('href')
print("Text:", text)
print("Link:", href)
print("-" * 40)
else:
print("Failed to retrieve webpage. Status code:", response.status_code)
```

Toggle Gemini

Conclusion:-

Web scraping using tools like Requests and BeautifulSoup enables efficient data extraction, but must always be done responsibly, ethically, and legally. This practical demonstrated how to programmatically fetch web content and parse it to extract specific information, such as links. Web scraping is a powerful technique for data collection from the internet, vital for tasks like market research, data analysis, and content aggregation, provided it adheres to legal frameworks and website policies.

Toggle Gemini